ТРАНЗАКЦІЇ ТА ОПТИМІЗАЦІЯ SQL

Справа про зниклі 50 грн: рівні ізоляції, аномалії, локи, черга на SKIP LOCKED і повільний запит під EXPLAIN

Заняття №14

Node.js Pro

Лектор
Кирило Орлов

r_d

СПРАВА №1414: З РАХУНКУ ЗНИКЛИ 50 ГРН

Перша половина — розслідування	Друга половина — локи на службі
крок 1 · відтворюємо: апдейт зникає	крок 5 · deadlock і порядок локів
крок 2 · READ COMMITTED: три аномалії	крок 6 · черга на SKIP LOCKED
крок 3 · знімок RR · 40001 · retry	кроки 7-8 · advisory · savepoints · EXPLAIN
крок 4 · FOR UPDATE / NOWAIT	40001 і 40P01 — не баги, а команда «повтори»

НАСКРІЗНА НИТКА

Один сценарій на весь вечір: alice = 100, A додає +50, B знімає −30, чекаємо 120. Кожен крок — окремий процес проти живого Postgres 16 на порту 21400; усі числа й коди помилок у деці — з реальних запусків.

ПЛАН ЗАНЯТТЯ



- Відтворюємо втрату 50 грн: read-modify-write
- Що бачить транзакція: READ COMMITTED і аномалії
- REPEATABLE READ: знімок, 40001, retry-патерн
- Write skew — і чому його ловить лише SERIALIZABLE
- Локи: FOR UPDATE, NOWAIT, deadlock, SKIP LOCKED
- Advisory locks для крону + savepoints для батчу
- Повільний запит: EXPLAIN і 5 переписувань · ДЗ

СЬОГОДНІ ПОБАЧИТЕ НА ВЛАСНІ ОЧІ

01. lost update

фінал 70 замість 120: апдейт зник — і жодної помилки

02. код 40001

БД сама командує «повтори»: retry через 68 мс → фінал 120

03. write skew

обидва COMMIT пройшли — а інваріант зламано

04. deadlock

40P01 з іменами процесів у DETAIL + цикл у docker logs

05. SKIP LOCKED

черга воркерів: 1238 мс → 337 мс, розподіл 3/3/3/3

06. EXPLAIN

37.9 → 0.3 мс без жодного нового індексу + чесний програш

ПРОБЛЕМА: ДВОЄ ПИШУТЬ В ОДИН РЯДОК

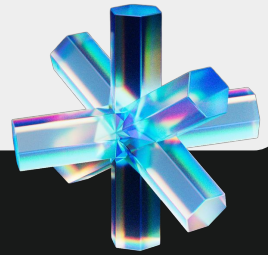
read-modify-write: читаємо в JS, рахуємо в JS, пишемо назад

між read і write — вікно: 150 мс «подумати» вистачає

BEGIN/COMMIT навколо двох запитів НЕ лікує: читання старе

у проді це баланс, залишок на складі, лічильник спроб

Сьогодні без моків: два клієнти pg проти живого Postgres 16. Кожну аномалію відтворюємо детерміновано — і підбираємо штатний інструмент.



```
ТАЙМЛАЙН ГОНКИ

A           B
—           —
SELECT 100
        SELECT 100
(думає)
UPDATE 150
        (думає)
        UPDATE 70 ✗

БД:      70
чекали: 120

_________________

запис A затерто
значенням, яке B
порахував зі
СТАРОГО читання
```

КРОК 1 · LOST UPDATE: ЗЛОЧИН ВІДТВОРЕНО

```
// src/01-lost-update.ts — read-modify-write у застосунку
async function readModifyWrite(
  tag: string, delta: number, thinkMs: number,
): Promise<void> {
  const c = await pool.connect();
  try {
    const r = await c.query<{ balance: string }>(
      `SELECT balance FROM accounts WHERE owner = 'alice'`);
    const seen = Number(r.rows[0]?.balance); // A:100 B:100
    await sleep(thinkMs);   // «думаємо»: знижка, сервіс…
    await c.query(`UPDATE accounts SET balance = $1
                   WHERE owner = 'alice'`, [seen + delta]);
  } finally { c.release(); }
}
await Promise.all([
  readModifyWrite('A', +50, 150),   // читає й пише першим
  (async () => { await sleep(50);
    await readModifyWrite('B', -30, 250); })(), // останній
]);
// [B] записав balance = 100 + (-30) = 70 ← затер 150 від A
// фінал у БД: 70 (очікували 120) — апдейт A (+50) ЗНИК
```

Сценарій справи: A додає +50, B знімає −30

1 · SELECT → 100: обидва бачать одне число

2 · sleep(thinkMs) — «думаємо»: знижка, інший сервіс

3 · UPDATE $1 — пишемо число, пораховане у JS

вивід: фінал у БД 70, очікували 120

урок: помилки НЕМАЄ — дані тихо збрехали

«ІНВЕНТАРИЗАЦІЯ СКЛАДУ»: ІЗОЛЯЦІЯ ЯК АНАЛОГІЯ

ЧАСТИНА POSTGRES	АНАЛОГІЯ (РЕВІЗІЯ)
• транзакція — сеанс роботи з даними • знімок (snapshot) даних • READ COMMITTED — знімок на кожен запит • REPEATABLE READ — знімок на транзакцію • SERIALIZABLE — «наче по черзі»	• ревізор рахує товар на складі • фото полиць, по якому він рахує • нове фото перед кожним поглядом • одне фото на всю ревізію • ревізії — ніби нікого поруч не було

Де аналогія ламається: фото захищає ЧИТАННЯ, а не запис. Двоє ревізорів за старими фото щось спишуть — і фото не попередить (це write skew). І фото не заважає двом схопити ту саму коробку — це вже про локи, не про знімки.



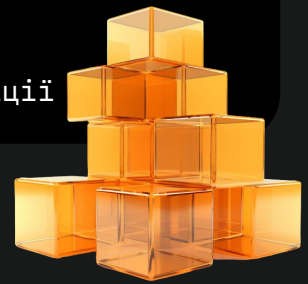
КРОК 2 · READ COMMITTED: ЩО ВИДНО, А ЩО НІ

- дефолт Postgres: кожен SELECT бере свіжий знімок
- dirty read неможливий: незакомічене не видно НІКОМУ
- non-repeatable: два SELECT в одній транзакції — різні числа
- phantom: повторний COUNT ловить «привидів» — нові рядки
- для ОДНОГО запиту цього досить: вікна між SELECT нема
- звіт із кількох SELECT їде по «пливучих» даних → крок 3

```
READ COMMITTED · РЕАЛЬНИЙ ПРОГІН КРОКУ 2

[A] UPDATE alice = 999 (без COMMIT)
[B] SELECT alice → 100
      dirty read: НІ
[A] ROLLBACK · BEGIN; SELECT alice → 100
[B] UPDATE alice = 150 (автокоміт)
[A] SELECT alice ще раз → 150
      non-repeatable: ТАК
[A] BEGIN; COUNT(*) → 2
[B] INSERT carol (автокоміт)
[A] COUNT(*) ще раз → 3
      phantom: ТАК

кожен SELECT = СВІЖИЙ знімок,
навіть усередині однієї транзакції
```



```
// src/lib/db.ts — retry: 40001 serialization · 40P01 deadlock
const RETRYABLE = new Set(['40001', '40P01']);
export async function withRetry<T>(
  label: string, isolation: 'REPEATABLE READ' | 'SERIALIZABLE',
  fn: (c: PoolClient) => Promise<T>, maxAttempts = 5,
): Promise<T> {
  for (let attempt = 1; ; attempt += 1) {
    try {
      return await tx(fn, isolation);  // BEGIN … COMMIT/ROLLBACK
    } catch (e) {
      const err = asPgError(e);
      if (err.code !== undefined && RETRYABLE.has(err.code)
          && attempt < maxAttempts) {
        const backoff =
          Math.round(2 ** attempt * 25 + Math.random() * 25);
        await sleep(backoff);        // експонента + jitter
        continue;                    // повтор транзакції ЦІЛКОМ
      }
      throw e;    // інший код = справжня помилка — НЕ ретраїмо
    }
  }
}
// [B] спроба 1 впала: 40001 → retry через 68 мс → фінал: 120
```



```
// src/03-repeatable-read.ts — write skew: інваріант «черговий ≥ 1»
const goHome = async (c: PoolClient, who: string) => {
  const n = await c.query<{ n: string }>(
    `SELECT count(*) AS n FROM oncall WHERE on_duty`);
  // «чергових зараз: 2 ≥ 2 → можна йти» — бачать ОБИДВА
  await c.query(`UPDATE oncall SET on_duty = false
                 WHERE doc = $1`, [who]);  // кожен пише СВІЙ рядок
};
await a.query('BEGIN ISOLATION LEVEL REPEATABLE READ');
await b.query('BEGIN ISOLATION LEVEL REPEATABLE READ');
await goHome(a, 'alice');
await goHome(b, 'bob');
await a.query('COMMIT');  await b.query('COMMIT');  // обидва ОК?!
// чергових лишилось: 0 — інваріант зламано.
// Рядки РІЗНІ → RR конфлікту НЕ бачить: знімок захищає
// читання, а не бізнес-правило. Це і є write skew.

// те саме під SERIALIZABLE:
// [A] COMMIT ok
// [B] COMMIT → 40001: could not serialize access due to
//     read/write dependencies among transactions
// SSI ловить skew на COMMIT — і лікування знову withRetry
```



РІВНІ ІЗОЛЯЦІЇ: ЩО КУПУЄШ І ЩО ПЛАТИШ

READ COMMITTED — дефолт, дешево; аномалії кроку 2 — твоя проблема

REPEATABLE READ — стабільний знімок; ціна: конкурентний запис → 40001

SERIALIZABLE (SSI) — ловить і write skew; ціна: більше 40001, вже на COMMIT

Рівень обирають ПАРОЮ з retry: RR/SERIALIZABLE без withRetry — це прод, що падає «інколи, не відтворюється». І навпаки: 40001/40P01 у логах — не шум, а нагадування, що повтор забули. Рівень задається на транзакцію: BEGIN ISOLATION LEVEL ... — глобально міняти не треба





```
// src/04-for-update.ts — песимістичні ліки від lost update
// 4.1 SELECT … FOR UPDATE: конкурент ЧЕКАЄ і читає вже свіже
await tx(async (c) => {
  const r = await c.query<{ balance: string }>(
    `SELECT balance FROM accounts WHERE owner='alice' FOR UPDATE`);
  const seen = Number(r.rows[0]?.balance);
  await sleep(thinkMs);                    // «думаємо» ПІД локом
  await c.query(`UPDATE accounts SET balance = $1
                 WHERE owner = 'alice'`, [seen + delta]);
});
// [B] FOR UPDATE → balance = 150 (чекав на лок 96 мс) → фінал 120

// 4.2 ще краще: рахує САМА БД — вікна read→write просто немає
await pool.query(`UPDATE accounts SET balance = balance + $1
                  WHERE owner = 'alice' AND balance + $1 >= 0
                  RETURNING balance`, [delta]);

// 4.3 чекати не можна (запит з UI, health-check) — NOWAIT
await pool.query(`SELECT * FROM accounts
                  WHERE owner = 'alice' FOR UPDATE NOWAIT`);
// [C] → 55P03: could not obtain lock on row — миттєва відмова
```

КРОК 5 · DEADLOCK: ЧИТАЄМО СПРАВУ 40P01

DETAIL

Process 148 waits for ShareLock on transaction 780; blocked by process 147 — обидва учасники названі поіменно

docker logs

log_lock_waits=on: у лозі сервера видно обидва UPDATE циклу — хто кого чекав

ordered locks

всі беруть локи в ОДНОМУ порядку (ORDER BY id) — цикл стає неможливим

Чому це важливо саме зараз

Deadlock не лагодиться if-ом у коді: Postgres сам обирає жертву після deadlock_timeout (у compose 200 мс для показу, дефолт 1 с). Жертва отримує 40P01 — а він уже в RETRYABLE нашого withRetry. Профілактика дешевша за лікування: порядок локів.

П'ЯТЬ КОДІВ: ВІДПОВІДІ БД, А НЕ БАГИ

Код	Що сталось	Що робити
40001 serialization_failure	знімок RR/SSI застарів проти конкурента	withRetry: повторити цілком
40P01 deadlock_detected	взаємне чекання, Postgres обрав жертву	retry + один порядок локів
55P03 lock_not_available	FOR UPDATE NOWAIT не отримав лок одразу	не чекати: 409 / «зайнято»
25P02 in_failed_sql_transaction	після помилки транзакція «отруєна»	ROLLBACK або savepoint
23514 check_violation	дані порушили CHECK — бізнес-правило	НЕ ретраїти: лагодити дані

ДЕ ГОНКИ ХОВАЮТЬСЯ У ЖИВОМУ КОДІ

01. read-modify-write через застосунок: баланс, склад, ліміти — крок 1

02. «перевірив умову — зробив дію» на РІЗНИХ рядках — write skew

03. крон на кількох інстансах без координації — advisory lock

04. воркери розбирають задачі з таблиці — SKIP LOCKED, крок 6

05. детектор: 40001 / 40P01 / lock waits у логах — сигнал, не шум

КАРТА РІШЕНЬ: ВІД СИМПТОМУ ДО ІНСТРУМЕНТУ

двоє пишуть той самий рядок	→	UPDATE … RETURNING або SELECT … FOR UPDATE
чекати не можна (UI, probe)	→	FOR UPDATE NOWAIT → 55P03
звіт із кількох SELECT	→	REPEATABLE READ
інваріант на КІЛЬКОХ рядках	→	SERIALIZABLE
40001 / 40P01 у логах	→	withRetry — повтор, не if
черга задач на воркерів	→	FOR UPDATE SKIP LOCKED
один крон на N інстансів	→	pg_try_advisory_lock(key)
батч, де 1 рядок отруйний	→	SAVEPOINT + ROLLBACK TO
правильний, але повільний	→	EXPLAIN ANALYZE → перепиши → EXPLAIN ще раз

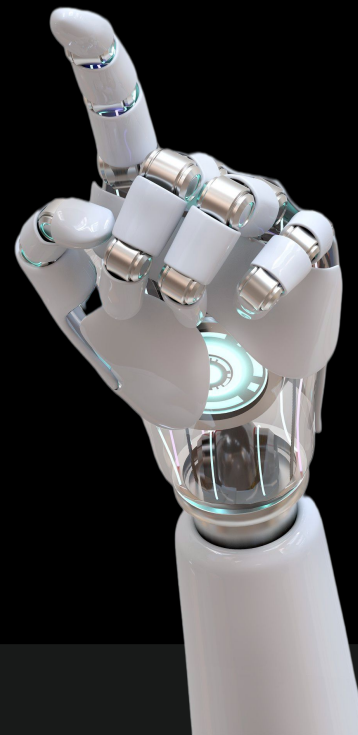
—— ТУТ ЗАКІНЧУЮТЬСЯ ЛОКИ, ПОЧИНАЄТЬСЯ ПЛАНЕР ————————



КРОК 6 · ЧЕРГА В POSTGRES: ЦИФРИ ПРОГОНУ

Варіант	**12 задач × 100 мс**	**Розподіл по 4 воркерах**
послідовно (теорія)	1200 мс	один воркер тягне все
ідеал (теорія)	300 мс	3 / 3 / 3 / 3
FOR UPDATE	1238 мс — як послідовно	нерівно: зазвичай хтось без задач
FOR UPDATE SKIP LOCKED	337 мс — ~ідеал	3 / 3 / 3 / 3 — чесно
другий прогін SKIP LOCKED	334 мс — стабільно	3 / 3 / 3 / 3 — двічі: 0

```
// src/06-skip-locked.ts — черга задач прямо в Postgres
async function claimOne(worker: string, skipLocked: boolean) {
  return tx(async (c) => {
    const r = await c.query<{ id: string }>(
      `SELECT id FROM jobs WHERE status = 'new'
       ORDER BY id LIMIT 1
       FOR UPDATE ${skipLocked ? 'SKIP LOCKED' : ''}`);
    if (r.rows.length === 0) return 'empty';
    await sleep(WORK_MS);   // «обробка»: лок живе всю транзакцію
    await c.query(`UPDATE jobs SET status = 'done', worker = $1,
                   processed = processed + 1 WHERE id = $2`,
                  [worker, r.rows[0]!.id]);
    return 'done';
  });
}
// без SKIP LOCKED: усі 4 воркери чекають ОДИН рядок:
//   [FOR UPDATE ] 1238 мс · розподіл нерівний, хтось голодує
// зі SKIP LOCKED: залоковане пропустив, узяв вільне:
//   [SKIP LOCKED]  337 мс · w1=3 w2=3 w3=3 w4=3 · двічі: 0
// впав воркер до COMMIT → лок зник, задача ПОВЕРНУЛАСЬ у чергу
// «порожньо» = «вільних нема»: хвіст добирають сусіди
```



КРОК 7 · ADVISORY LOCK І SAVEPOINTS

крон на 2 інстансах: pg_try_advisory_lock(1414) → true / false

лочимо ІМ'Я (число), не рядок — таблички-«прапорця» не існує

інстанс-2: false → пропускає запуск; після unlock — його черга

батч 5 платежів, №3 ламає CHECK: 23514 → 25P02 → у БД 0 рядків

SAVEPOINT на рядок + ROLLBACK TO: COMMIT пройшов, у БД 4 з 5

так ORM роблять «вкладені» транзакції; savepoint не безплатний

КРОК 8 · П'ЯТЬ ПЕРЕПИСУВАНЬ, ЧИСЛА МОГО ЗАПУСКУ

ПРИЙОМ

- date(created_at) → діапазон >= і <
- CTE MATERIALIZED → інлайн (дефолт з PG12)
- JOIN до пагінації → deferred join
- OFFSET 200000 → keyset-пагінація
- «агрегуй до JOIN» — контрприклад

ЕФЕКТ (EXPLAIN ANALYZE)

- 37.9 → 0.3 мс — індекс нарешті видно (Index Only Scan)
- 70.2 → 0.03 мс — предикат доїхав до індексу
- 106.5 → 54.4 мс — приєднуємо лише 20 рядків, не всі
- 16.2 → 0.3 мс — стрибок по індексу замість 200k читань
- ПРОГРАШ 46.5 → 77.9 мс — планер сам уже паралелив

```
-- src/08-slow-query.ts · 8.4: сторінка №10 000 (по 20 рядків)
-- ДО: OFFSET читає і ВИКИДАЄ 200 000 рядків на кожну сторінку
SELECT id, created_at FROM orders
ORDER BY created_at, id OFFSET 200000 LIMIT 20;
-- ⏱️ 16.2 мс — і росте разом із номером сторінки

-- ПІСЛЯ: keyset — курсор = (created_at, id) останнього рядка
SELECT id, created_at FROM orders
WHERE (created_at, id) > ($1, $2)     -- row-value порівняння
ORDER BY created_at, id LIMIT 20;
-- ⏱️ 0.3 мс: Index Only Scan стрибнув одразу на місце
-- ціна: «одразу на сторінку 573» не можна; курсор їде в API

-- 8.1 той самий принцип «дай індексу шанс»:
--   date(created_at) = '2026-05-15'    37.9 мс  Seq Scan
--   created_at >= … AND created_at < …  0.3 мс  Index Only

-- 8.5 контрприклад: «агрегуй до JOIN» ПРОГРАЛО 46.5 → 77.9 мс
--   планер сам розбив v1 на parallel partial aggregate,
--   а «розумний» підзапит v2 цю можливість відрізав.
--   Мораль кроку: EXPLAIN до І після. Щоразу.
```

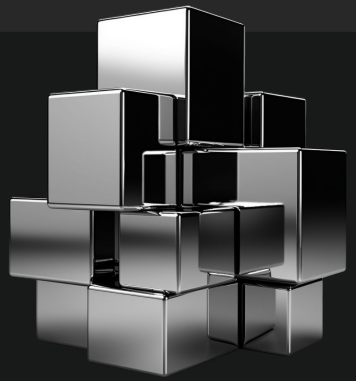


ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

розподілені транзакції: two-phase commit — лише концепт у волті; SAGA та outbox — у блоці мікросервісів. Сьогодні чесно один Postgres

індекси НЕ чіпали свідомо: вся оптимізація кроку 8 — переписуванням, індекси з L#12 стояли весь час · пул, PgBouncer, репліки, бекапи → L#15 · MVCC-нутрощі й vacuum — було на L#12 · транзакції в ORM — L#13

⚠ Node v22.21.1 · pg 8.22.0 · typescript 6.0.3 · Postgres 16.13 (образ postgres:16). Тайминги EXPLAIN на іншій машині будуть інші — порядок ефекту той самий. deadlock_timeout=200ms у compose — для швидкого показу, дефолт 1 с



ПІДСУМКИ + МІСТ ДО L#15

АНОМАЛІЇ

lost update, non-repeatable, phantom, write skew — кожну відтворили детерміновано і кожній підібрали штатний інструмент

КОДИ БД

40001 і 40P01 — команда «повтори цілком»: withRetry; deadlock лікує порядок локів; NOWAIT і SKIP LOCKED — коли чекати не можна

ДАЛІ

L#15 — коли один Postgres закінчується: пул з'єднань, PgBouncer, репліки, шардінг, бекапи і DR

ДЗ #14

7 балів (PROJECT MEDIUM) · дедлайн 15.09 · для курсового: оформлення замовлення з декрементом stock БЕЗ мінусів під конкурентним навантаженням + worker pool через SKIP LOCKED + retry на 40001

EXPLAIN

правильний ≠ швидкий: EXPLAIN до → переписав → EXPLAIN після. Контрприклад 8.5 нагадує: правило без вимірювання — не правило

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ